

Министерство образования Республики Беларусь

Учреждение образования
БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИНФОРМАТИКИ И РАДИОЭЛЕКТРОНИКИ

Факультет компьютерных систем и сетей
Кафедра информатики
Дисциплина: Методы защиты информации

ОТЧЕТ
по лабораторной работе №8
на тему

Стеганографические методы

Студент

Т. П. Власенко

Проверил

А. В. Герчик

Минск 2024

СОДЕРЖАНИЕ

1 Цель работы	3
2 Теоретические сведения	4
3 Результат работы программы.....	5
Вывод.....	7
Приложение А (обязательное) Листинг программного кода	8

1 ЦЕЛЬ РАБОТЫ

Данная лабораторная работа нацелена на разработку программного средства для сокрытия и извлечения текстовых сообщений в (из) JPEG изображениях с использованием метода сокрытия в частотной области. Этот метод основан на использовании дискретного косинусного преобразования, которое позволяет представить изображение в частотной области и внести в него скрытую информацию без существенного воздействия на визуальное восприятие изображения.

2 ТЕОРЕТИЧЕСКИЕ СВЕДЕНИЯ

Блок-схема алгоритма представлена на рисунке 1.

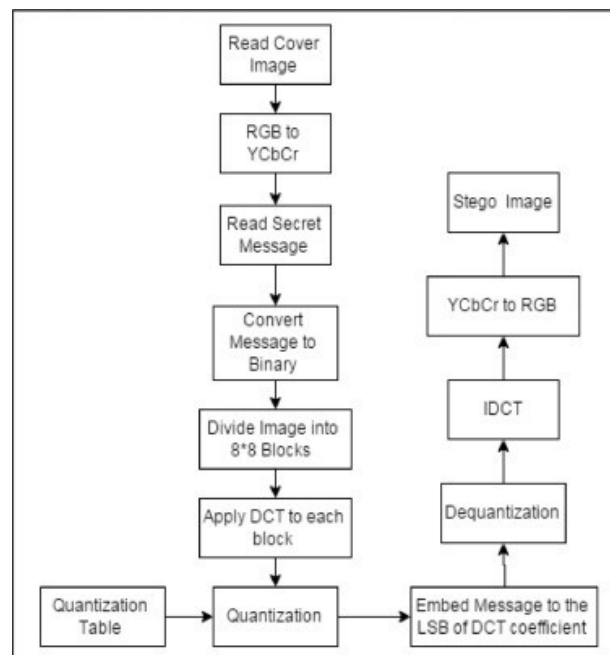


Рисунок 1 – Блок-схема алгоритма

В данном способе изображение разделяется на блоки 8x8 пикселей, каждый из которых используется для шифрования одного бита сообщения. Шифрование начинается с произвольного подбора блока, используемого для шифрования i -го бита сообщения. Затем для подобранного блока применяют ДКП.

LSB (Least Significant Bit, наименьший значащий бит (НЗБ)) — суть этого метода заключается в замене последних значащих битов в контейнере (изображения, аудио или видеозаписи) на биты скрываемого сообщения. Разница между пустым и заполненным контейнерами должна быть не ощутима для органов восприятия человека.

Скрытие: Исходное изображение-контейнер разделяется на блоки по 8x8 пикселей, к которым применяется DCT: $F_{\{i\}}[a,b]=DCT(O_{\{i\}}[a,b])$. Из каждого коэффициента матрицы $F_{\{i\}}$ выделяются наименее значимые биты и заменяются на биты скрываемого сообщения[.

Извлечение: Изображение-контейнер разделяется на блоки $O_{\{i\}}$ по 8x8 пикселей, к которым применяется DCT: $F_{\{i\}}[a,b]=DCT(O_{\{i\}}[a,b])$. Из каждого коэффициента матрицы $F_{\{i\}}$ выделяются наименее значимые биты и объединяются, восстанавливая скрытое сообщение.

3 РЕЗУЛЬТАТ РАБОТЫ ПРОГРАММЫ

Исходное изображение представлено на рисунке 2



Рисунок 2 – Входные данные

Исходный текст передаётся в файле input.txt.

```
1  привет hello 1234 !@#!@$!  
2  
3      14123 ываыф  
4  
5      asdfa
```

Рисунок 3 – Шифруемая информация

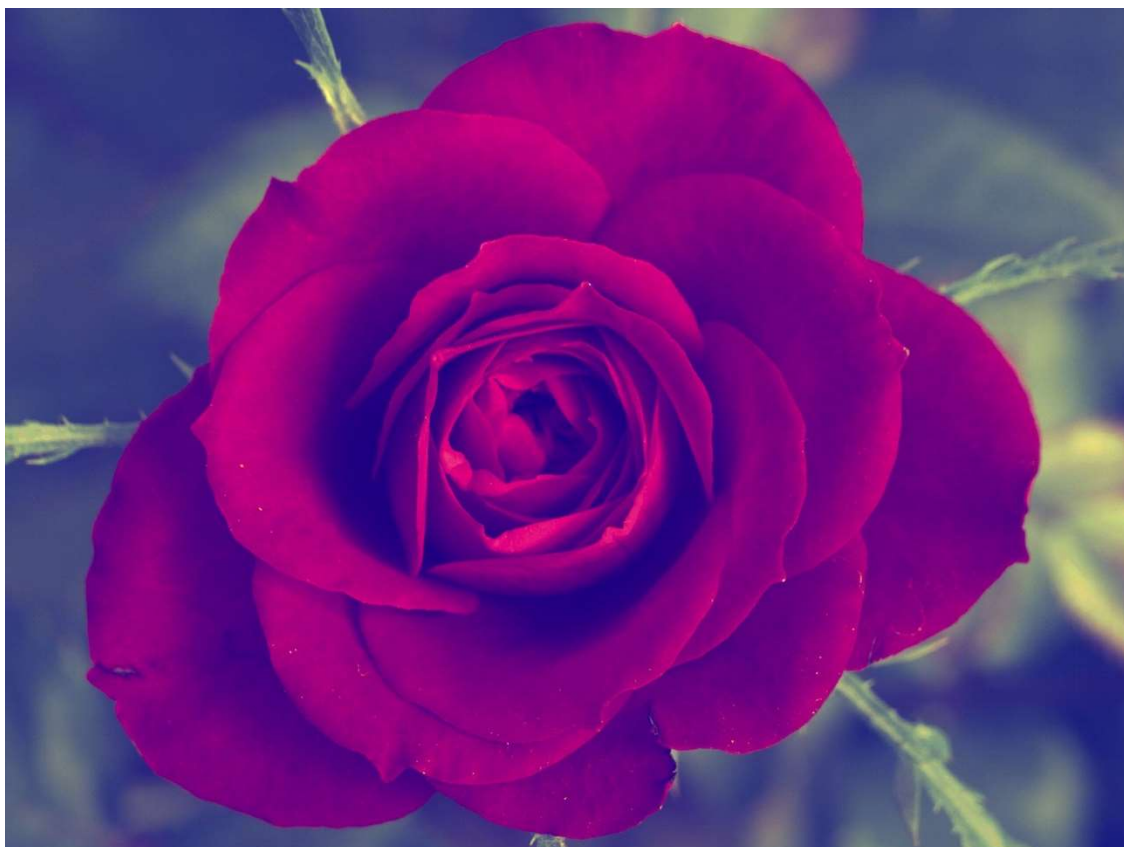


Рисунок 4 – Зашифрованное изображение

ВЫВОД

В рамках данной лабораторной работы было разработано программное средство для сокрытия и извлечения текстовых сообщений в (из) JPEG изображениях на основе метода сокрытия в частотной области с использованием дискретного косинусного преобразования.

ПРИЛОЖЕНИЕ А

(обязательное)

Листинг программного кода

Файл main.py

```
from __future__ import print_function

import warnings

import cv2
import itertools
import numpy as np

quant = np.array([[16, 11, 10, 16, 24, 40, 51, 61],
                  [12, 12, 14, 19, 26, 58, 60, 55],
                  [14, 13, 16, 24, 40, 57, 69, 56],
                  [14, 17, 22, 29, 51, 87, 80, 62],
                  [18, 22, 37, 56, 68, 109, 103, 77],
                  [24, 35, 55, 64, 81, 104, 113, 92],
                  [49, 64, 78, 87, 103, 121, 120, 101],
                  [72, 92, 95, 98, 112, 100, 103, 99]])

class DCT:
    def __init__(self, imPath):
        self.imPath = imPath
        self.message = None
        self.bitMess = None
        self.oriCol = 0
        self.oriRow = 0
        self.numBits = 0

    def DCTEn(self, secret, outIm):
        img = self.loadImage()
        if img is None:
            print("Error: File not found!")
            return

        self.message = str(len(secret)) + '*' + secret
        self.bitMess = self.toBits()

        row, col = img.shape[:2]
        self.oriRow, self.oriCol = row, col

        if ((col / 8) * (row / 8) < len(secret)):
            print("Error: Message too large to encode in image")
            return

        if row % 8 != 0 or col % 8 != 0:
```

```

        img = self.addPadd(img, row, col)

    row, col = img.shape[:2]

    bImg, gImg, rImg = cv2.split(img)

    bImg = np.float32(bImg)

    imgBlocks = [np.round(bImg[j:j + 8, i:i + 8] - 128) for (j, i) in
                  itertools.product(range(0, row, 8), range(0, col, 8))]

    dctBlocks = [np.round(cv2.dct(img_Block)) for img_Block in imgBlocks]

    quantizedDCT = [np.round(dct_Block / quant) for dct_Block in
                    dctBlocks]

    messIndex = 0
    letterIndex = 0

    for quantizedBlock in quantizedDCT:
        DC = quantizedBlock[0][0]
        DC = np.uint8(DC)
        DC = np.unpackbits(DC)
        DC[7] = self.bitMess[messIndex][letterIndex]
        DC = np.packbits(DC)

        DC = np.float32(DC)
        DC = DC - 255
        with warnings.catch_warnings():
            warnings.filterwarnings("ignore",
category=DeprecationWarning)
            quantizedBlock[0][0] = DC

        letterIndex = letterIndex + 1
        if letterIndex == 8:
            letterIndex = 0
            messIndex = messIndex + 1
            if messIndex == len(self.message):
                break

    sImgBlocks = [quantizedBlock * quant +
                  128 for quantizedBlock in quantizedDCT]

    sImg = []
    for chunkRowBlocks in self.chunks(sImgBlocks, col / 8):
        for rowBlockNum in range(8):
            for block in chunkRowBlocks:
                sImg.extend(block[rowBlockNum])
    sImg = np.array(sImg).reshape(row, col)

    sImg = np.uint8(sImg)

    sImg = cv2.merge((sImg, gImg, rImg))
    cv2.imwrite(outIm, sImg)

```

```

        return sImg

def DCTDe(self):
    img = cv2.imread(self.imPath, cv2.IMREAD_UNCHANGED)
    row, col = img.shape[:2]

    messSize = None
    messageBits = []
    buff = 0

    bImg, gImg, rImg = cv2.split(img)
    bImg = np.float32(bImg)

    imgBlocks = [bImg[j:j + 8, i:i + 8] - 128 for (j, i) in
itertools.product(range(0, row, 8),
range(0, col, 8))]
    quantizedDCT = [img_Block / quant for img_Block in imgBlocks]
    i = 0
    for quantizedBlock in quantizedDCT:
        DC = quantizedBlock[0][0]
        DC = np.uint8(DC)
        DC = np.unpackbits(DC)
        if DC[7] == 1:
            buff += (0 & 1) << (7 - i)
        elif DC[7] == 0:
            buff += (1 & 1) << (7 - i)
        i = 1 + i
        if i == 8:

            messageBits.append(chr(buff))
            buff = 0
            i = 0

            if messageBits[-1] == '*' and messSize is None:
                try:
                    messSize = int(''.join(messageBits[:-1]))
                except:
                    pass
            if len(messageBits) - len(str(messSize)) - 1 == messSize:
                return ''.join(messageBits)[len(str(messSize)) + 1:]

    return ''

def chunks(self, l, n):
    m = int(n)
    for i in range(0, len(l), m):
        yield l[i:i + m]

def loadImage(self):
    img = cv2.imread(self.imPath, cv2.IMREAD_UNCHANGED)

    if img is None:
        return None

```

```

        return img

    def addPadd(self, img, row, col):
        img = cv2.resize(img, (col + (8 - col % 8), row + (8 - row % 8)))
        return img

    def toBits(self):
        bits = []

        for char in self.message:
            binval = bin(ord(char))[2:].rjust(8, '0')

            bits.append(binval)

        self.numBits = bin(len(bits))[2:].rjust(8, '0')
        return bits

DCTEncode = DCT('input.jpg')
with open("input.txt", "r", encoding="utf-8") as f:
    message_to_decode = f.read()
print("input:\n", message_to_decode)
DCTEncode.DCTEn(message_to_decode, 'output.jpg')

DCTDecode = DCT('output.jpg')
message_to_decoded = DCTDecode.DCTDe()
print('output:\n', message_to_decode)

```